GIT & GITHUB PRACTICAL ASSIGNMENT

Name: Muhammed Fazinnaj
Project: Personal Portfolio Website

1. Introduction

This assignment demonstrates the practical implementation of Git and GitHub concepts such as version control, commits, branching, merging, and conflict resolution. A personal portfolio website was developed using HTML and CSS.

2. Repository and Branch Structure

Three feature branches were created: feature/homepage for homepage development, feature/styling for CSS styling, and feature/navbar for navigation links and footer development. Each branch contained at least two commits.

3. Merge Operations

A fast-forward merge was performed by merging the homepage branch into main. A three-way merge was performed using --no-ff to merge the styling branch and create a separate merge commit.

```
"Initial commit: add README and gitignore"
[main (root-commit) 8445a48] Initial commit: add README and gitignore
 2 files changed, 36 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
```

4. Merge Conflict Resolution

A merge conflict was deliberately created by modifying the same README heading on two branches. Git reported a conflict because the changes were different. The conflict was resolved by selecting the preferred heading, removing the conflict markers, and committing the resolved file.

```
PS Desktop\fazi-git-assignment> git merge feature/navbar
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS Desktop\fazi-git-assignment>
```

```
<<<<<<< HEAD (Current Change)
# Muhammed Fazinnaj's Portfolio Project
=======
# GitHub Portfolio Assignment
>>>>>>> feature/navbar (Incoming Change)
```

```
PS Desktop\fazi-git-assignment> git commit -m "Merge navbar branch and resolve
README conflict"
[main 10e60a4] Merge navbar branch and resolve README conflict
```

5. Part B — Repository Exploration

The Hello-World repository was cloned and explored using git log --oneline --graph --all, git branch -a, git log -p -3, and git blame README. These commands helped identify commit history, branches, file modifications, and contributors.

Reflection Questions

Q1. What is the difference between a fast-forward merge and a three-way merge?

A fast-forward merge moves the main branch pointer forward without creating a separate merge commit. A three-way merge combines diverged branches and creates a merge commit. In this project, the homepage branch used a fast-forward merge, while the styling branch used a three-way merge.

Q2. How was the merge conflict resolved?

The conflict occurred because the same README heading was modified differently on two branches. The preferred heading was selected, the conflict markers were removed, and the resolved file was committed.

Q3. How was the cloned repository different from the original project?

The personal portfolio repository contained three feature branches organized around specific development tasks. The cloned repository contained an existing commit history created by other developers, which could be explored using Git commands.

Q4. What problems can force-pushing create?

Force-pushing rewritten history can overwrite remote commits. Developers who have already pulled the previous commits may experience diverged histories, making collaboration and synchronization difficult.

6. Conclusion

This assignment provided practical experience in repository management, branching, merging, conflict resolution, and exploring Git history. It demonstrated how Git and GitHub help developers track changes and collaborate effectively.